

# Logging Error Messages and Warning Levels

## Task

Write a Java application that demonstrates logging error messages and warning levels using SLF4J.

## Step 1: Add Dependencies

Add the following dependencies to your `pom.xml` file.

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>

<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version>
</dependency>
```

## Step 2: Create Java Class

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {

        logger.error("This is an error message.");

        logger.warn("This is a warning message.");
```

```
}  
}
```

## Expected Output

ERROR LoggingExample - This is an error message.

WARN LoggingExample - This is a warning message.

# Parameterized Logging

## Task

Write a Java application that demonstrates parameterized logging using SLF4J.

## Step 1: Add Dependencies

Add the following dependencies to your `pom.xml` file.

```
<dependency>  
  <groupId>org.slf4j</groupId>  
  <artifactId>slf4j-api</artifactId>  
  <version>1.7.30</version>  
</dependency>  
  
<dependency>  
  <groupId>ch.qos.logback</groupId>  
  <artifactId>logback-classic</artifactId>  
  <version>1.2.3</version>  
</dependency>
```

## Step 2: Create Java Class

```
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
  
public class ParameterizedLoggingExample {  
  
    private static final Logger logger =  
        LoggerFactory.getLogger(ParameterizedLoggingExample.class);  
  
    public static void main(String[] args) {
```

```
String name = "Alice";
int age = 22;
double salary = 50000.50;

logger.info("Employee Name: {}", name);

logger.info("Employee Name: {}, Age: {}", name, age);

logger.info("Employee Name: {}, Salary: {}", name, salary);

logger.warn("Employee {} has not submitted the report.", name);

logger.error("Unable to process salary for employee {}.", name);
}
}
```

## Expected Output

```
INFO Employee Name: Alice
INFO Employee Name: Alice, Age: 22
INFO Employee Name: Alice, Salary: 50000.5
WARN Employee Alice has not submitted the report.
ERROR Unable to process salary for employee Alice.
```

# Using Different Appenders

## Task

Write a Java application that demonstrates using different appenders with SLF4J.

## Step 1: Add Dependencies

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>

<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
```

```
<version>1.2.3</version>
</dependency>
```

## Step 2: Create **logback.xml**

Place this file inside the `src/main/resources` folder.

```
<configuration>

  <!-- Console Appender -->
  <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <!-- File Appender -->
  <appender name="file" class="ch.qos.logback.core.FileAppender">
    <file>app.log</file>

    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <root level="debug">
    <appender-ref ref="console"/>
    <appender-ref ref="file"/>
  </root>

</configuration>
```

## Step 3: Create Java Class

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AppenderExample {

  private static final Logger logger =
    LoggerFactory.getLogger(AppenderExample.class);

  public static void main(String[] args) {
```

```
    logger.debug("This is a DEBUG message.");

    logger.info("This is an INFO message.");

    logger.warn("This is a WARNING message.");

    logger.error("This is an ERROR message.");

    logger.info("Logs are displayed on the console and stored in app.log.");
}
}
```

## Expected Console Output

```
10:15:30.123 [main] DEBUG AppenderExample - This is a DEBUG message.
10:15:30.125 [main] INFO  AppenderExample - This is an INFO message.
10:15:30.126 [main] WARN  AppenderExample - This is a WARNING message.
10:15:30.127 [main] ERROR AppenderExample - This is an ERROR message.
10:15:30.128 [main] INFO  AppenderExample - Logs are displayed on the console and stored in
app.log.
```

## Generated **app.log**

```
10:15:30.123 [main] DEBUG AppenderExample - This is a DEBUG message.
10:15:30.125 [main] INFO  AppenderExample - This is an INFO message.
10:15:30.126 [main] WARN  AppenderExample - This is a WARNING message.
10:15:30.127 [main] ERROR AppenderExample - This is an ERROR message.
10:15:30.128 [main] INFO  AppenderExample - Logs are displayed on the console and stored in
app.log.
```